Introduction

Organizor is the backend API for managing tenant environments. It keeps tenant records in a central catalog database and provisions a separate PostgreSQL database for each tenant.

The current implementation focuses on tenant lifecycle operations:

- creating tenant catalog records;
- onboarding tenants by creating a tenant database and applying migrations through internal operations;
- resolving tenant context for tenant-scoped requests;
- upgrading tenant databases by running migrations;
- soft-deleting tenants before hard deletion;
- exposing health checks, OpenAPI JSON, and Scalar endpoint exploration in development.

Main Concepts

CONCEPT	DESCRIPTION
Tenant	A customer or isolated workspace tracked by `Id`, `Name`, `Slug`, `DatabaseName`, lifecycle status, and deletion metadata.
Catalog database	Central PostgreSQL database that stores all tenant records.
Tenant database	PostgreSQL database named from the tenant slug, currently using the `organizor_tenant_{slug}` convention.
Tenant resolution	Middleware step that reads `X-Tenant-Slug`, loads the tenant, and stores the current tenant in `ITenantContext`.
Provisioning workflow	Infrastructure workflow that creates the tenant database and applies tenant migrations.

Project Layout

PROJECT	RESPONSIBILITY
`Code7Dev.Organizor.Api`	ASP.NET Core host, controllers, DTOs, mappers, middleware, OpenAPI/Scalar, health checks, CORS, and logging setup.
`Code7Dev.Organizor.Application`	Tenant service contracts, tenant models, onboarding, lifecycle, upgrade, and catalog-facing application logic.
`Code7Dev.Organizor.Domain`	Tenant entity and `TenantStatus` enum.
`Code7Dev.Organizor.Infrastructure`	EF Core contexts, repository implementation, PostgreSQL connection string building, database creation/deletion, migration, and tenant context storage.

Documentation Boundaries

The documentation is split into two intentionally different areas:

AREA	PURPOSE
Conceptual docs	Explain behavior, workflows, configuration, security, local development, operations, and integration decisions.
Generated API reference	Lists C# namespaces, classes, interfaces, methods, and signatures generated from source.

Do not use the generated reference to explain workflows or product behavior. Keep that content in the conceptual articles so it stays readable and can include context, examples, and operational guidance.

Namespace Code7Dev.Organizor.Api.Authorization

Classes

AuthorizationPolicies

KeycloakRoleClaimsTransformation